Software Requirements Specification
AI Study Vault: RAG-Powered Learning Platform

Samuel Regan, Semyon Fox, Shreyansh Singh

February 16, 2026

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Key Stakeholders . . . 3
1.3 Scope, Objectives and Goals . . . 3
1.3.1 System Scope . . . 3
1.3.2 In Scope . . . 4
1.3.3 Out of Scope . . . 4
1.3.4 Key Objectives . . . 5
1.4 References and Acknowledgments . . . 5
1.5 Overview . . . 5

2 Overall Description **6**
2.1 Product Perspective . . . 6
2.2 Product Functions (High-Level) . . . 6
2.3 User Characteristics . . . 7
2.4 Constraints . . . 7
2.5 Assumptions and Dependencies . . . 7

3 Functional Requirements **8**
3.1 Feature Priority Tiers . . . 8
3.2 Functional Requirements Overview . . . 8
3.3 User Stories and Mapped Functional Requirements . . . 8
3.3.1 User Story 1: Complete Authentication Flow . . . 8
3.3.2 User Story 2: User Profile and Settings . . . 9
3.3.3 User Story 3: Onboarding and Course Information . . . 10
3.3.4 User Story 4: Create and Edit Markdown Notes . . . 10
3.3.5 User Story 5: Upload and Index PDFs . . . 10
3.3.6 User Story 6: RAG Chat with Citations . . . 11
3.3.7 User Story 7: Auto-Generate Quizzes . . . 12
3.3.8 User Story 8: Spaced Repetition Flashcards . . . 12
3.3.9 User Story 9: Offline Note Editing . . . 13
3.3.10 User Story 10: Canvas LMS Integration . . . 13
3.3.11 User Story 11: Study Calendar and Timetable . . . 14
3.3.12 User Story 12: Progress Analytics Dashboard . . . 14

3.3.13 User Story 13: Full Vault Export . . . 15
3.3.14 User Story 14: Quick Command Search (Cmd+K) . . . 15
3.3.15 User Story 15: Note Tagging and Linking . . . 15
3.3.16 User Story 16: Multi-Mode Search . . . 16
3.3.17 User Story 17: Share Materials with Study Groups . . . 16
3.3.18 User Story 18: Internationalization (i18n) Support . . . 17
3.3.19 User Story 19: Export Individual Notes/Data . . . 17
3.4 RAG Pipeline Architecture . . . 18
3.5 Functional Summary . . . 18

4 Non-Functional Requirements 19
4.1 Performance . . . 19
4.2 Security . . . 19
4.3 Reliability & Availability . . . 20
4.4 Usability . . . 20
4.5 Maintainability . . . 20

5 External Interface Requirements 21
5.1 User Interfaces . . . 21
5.1.1 Authentication Pages . . . 21
5.1.2 Settings Page . . . 21
5.1.3 Editor Interface . . . 21
5.1.4 RAG Chat Panel . . . 22
5.1.5 Quiz Interface . . . 22
5.1.6 Flashcard Interface . . . 22
5.1.7 Calendar/Timetable View . . . 22
5.2 API Interface . . . 22
5.3 Database Interface . . . 24
5.3.1 Storage Architecture . . . 24
5.3.2 Data Entities and Relationships . . . 24
5.3.3 Database Capabilities . . . 25
5.4 External Services . . . 25

6 Data & Legal Requirements 26
6.1 Data Handling . . . 26
6.2 Compliance . . . 26
6.3 Audit & Logging . . . 26

7 Verification and Acceptance 27
7.1 Testing Strategy . . . 27
7.2 Acceptance Criteria . . . 27
7.3 Deployment and CI/CD . . . 28

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **AI Study Vault**, an AI-powered study platform that combines portable Markdown notes with Retrieval-Augmented Generation (RAG) to provide intelligent learning from course materials.

The document establishes a shared contract between the development team and academic stakeholders for the project lifecycle. This follows an Agile approach, emphasizing user stories, acceptance criteria, and iterative delivery. The SRS focuses on *what* the system shall do and the quality attributes it shall meet, not on detailed design or implementation.

Intended Audience: Development team, academic stakeholder, future maintainers, and QA engineers.

1.2 Key Stakeholders

1. **Development Team**: Three developers with flexible work distribution across frontend, backend, and infrastructure. Primary specializations:
   - Database/AWS specialist (MariaDB, S3, RDS, deployment infrastructure)
   - RAG pipeline specialist (embeddings, vector search, LLM integration)
   - Shared responsibilities for frontend, API development, and testing
2. **Academic Stakeholder**: Approves scope, evaluates grading criteria, provides feedback and acceptance.
3. **End Users**: University students (primary) and study groups (secondary).

1.3 Scope, Objectives and Goals

1.3.1 System Scope

AI Study Vault is a web-based learning platform that allows students to:

- Create and manage Markdown notes, with offline-first support (PWA).
- Ingest and index PDFs into a vector database for semantic retrieval.
- Ask questions about their materials through an RAG-powered chat with citations.
- Generate adaptive quizzes and spaced-repetition flashcards.
- Track progress and weak topics with an analytics dashboard.
- Integrate with LMS platforms (e.g., Canvas) to sync deadlines.

1.3.2 In Scope

- Web-based Markdown note editor with offline support (PWA).
- PDF ingestion, text extraction, semantic chunking with MariaDB native vector storage.
- LMS integration (Canvas, Moodle) for read-only assignment and deadline sync (once daily automatic + on-demand).
- RAG chat: Ask questions about uploaded materials with cited answers.
- Adaptive quiz generation and difficulty scaling.
- Spaced repetition (SM-2 algorithm) for flashcard review.
- Calendar/timetable scheduling for tasks and study.
- Analytics: Track progress, weak topics, study time.
- Full vault export (all notes and metadata as downloadable archive).
- Bidirectional note linking and knowledge graph visualization.
- Flexible tagging system for note organization.
- Multi-mode search (keyword, semantic, hybrid).
- User authentication with email verification and password reset.
- User profile management and settings.
- Course information collection (university, degree, graduation year).
- Responsive design for desktop and tablet browsers.

1.3.3 Out of Scope

- Real-time simultaneous multi-user editing.
- Video/audio content processing and transcription.
- Native mobile apps (PWA only).
- Instructor administrative dashboards and grading workflows.
- Light mode / full WCAG compliance (dark mode only for MVP).
- Complex error recovery and retry logic (simplified error messages for MVP).
- Advanced monitoring and analytics platforms (Lighthouse audits only for MVP).

1.3.4 Key Objectives

1. Deliver a feature-complete MVP by end of semester (1–2 weeks before exams).
2. Demonstrate reproducible Docker Compose setup (works on a fresh environment).
3. Achieve 70%+ code coverage for core business logic (SM-2, RAG, chunking).
4. Enable 3-person parallel development via clear service boundaries and API contracts.
5. Deploy production builds automatically via AWS Amplify from the `prod` branch.
6. Maintain development within free tier limits or preset budget constraints.

1.4 References and Acknowledgments

- IEEE Std 830-1998: Software Requirements Specification.
- ISO/IEC/IEEE 29148:2018: Systems and Software Engineering.
- OpenAPI 3.1 Specification (REST API contracts).
- **Notea** (`https://github.com/QingWei-Li/notea`) – MIT License. This project uses Notea as a scaffold for the note-taking interface. We are deeply grateful to the Notea contributors for providing an excellent starting point.
- MariaDB Vector Documentation: `https://mariadb.com/docs/server/vector/`.

1.5 Overview

This SRS is organized as follows:

- **Section 2**: Overall description (product perspective, users, constraints, assumptions).
- **Section 3**: Functional requirements (user stories and formal FR identifiers).
- **Section 4**: Non-functional requirements (performance, security, reliability, usability, etc.).
- **Section 5**: External interface requirements (API, database, UI, external services).
- **Section 6**: Data and legal requirements (handling, retention, compliance, audit).
- **Section 7**: Verification and acceptance.
- **Appendix A**: Glossary and definitions.
- **Appendix B**: Use cases and scenarios.

2 Overall Description

2.1 Product Perspective

AI Study Vault is a new web-based application designed as a standalone Progressive Web App (PWA) that can be deployed in a cloud environment (e.g., AWS Lightsail + RDS). It integrates with external services (Canvas LMS, OpenAI API, S3-compatible storage) and uses MariaDB as a unified relational and vector database.

The platform follows a microservice-style architecture with clear service boundaries:

- **Frontend**: Next.js PWA client with Tailwind CSS for note editing, chat, quizzes, flashcards, and analytics.
- **Document Pipeline Service**: Handles PDF ingestion, text extraction, chunking, embedding generation, and vector storage.
- **Learning API Service**: Exposes REST APIs for RAG chat, quizzes, spaced repetition, analytics, and calendar.
- **Background Workers**: Process asynchronous jobs via BullMQ and Redis (e.g., PDF indexing, LMS sync).

2.2 Product Functions (High-Level)

At a high level, AI Study Vault provides the following major functions:

- Manage user accounts with email verification and password reset.
- Collect and manage user course information (university, degree, graduation year).
- Create, edit, organize, and sync Markdown notes.
- Ingest and index PDF documents into a vector store for semantic retrieval.
- Provide an RAG-based chat interface with cited answers.
- Generate and manage quizzes for self-assessment.
- Manage flashcards and spaced-repetition review schedules.
- Integrate LMS deadlines into a unified calendar and auto-schedule study sessions.
- Provide analytics and reporting on study progress and weak topics.
- Export entire vault (notes, metadata, settings) as a downloadable archive.
- Link notes bidirectionally and visualize knowledge graphs.
- Tag notes flexibly for organization and filtering.
- Search using keyword, semantic, or hybrid modes.

2.3 User Characteristics

- **Primary Users**: Undergraduate and postgraduate students familiar with web apps, note-taking tools, and LMS platforms.
- **Secondary Users**: Study group members and classmates accessing shared materials (nice-to-have feature).
- **Technical Proficiency**: Basic digital literacy; no programming experience required.
- **Accessibility Needs**: Support for keyboard navigation; dark mode by default.

2.4 Constraints

- **Technical Stack**:
  - Frontend: Next.js 14+ with Tailwind CSS, PWA capabilities
  - Backend: Node.js 18+
  - Database: MariaDB with native vector support
  - Storage: S3-compatible object storage
  - AI: OpenAI API for embeddings and chat completion
- **Deployment**:
  - Local: Docker Compose for development
  - Production: AWS Amplify auto-deploy from `prod` branch
  - Optional: AWS staging environment for pre-production testing
- **Budget**: Development must stay within free tier limits or preset budget constraints.
- **Storage Quotas**: Per-user storage quota of 5GB for MVP (configurable).
- **Time**: Development through end of semester 2 (1–2 weeks before exams for buffer).
- **Availability**: Team works few hours per day, 5 days per week.
- **Compliance**: Must not store LMS credentials or passwords in plain text; must adhere to basic GDPR practices.

2.5 Assumptions and Dependencies

- OpenAI API is available during the project; simplified error handling if unavailable (user-facing error, manual retry).
- S3-compatible storage and MariaDB instances are provisioned and reachable from app services.
- Canvas LMS and other LMS APIs are accessible for OAuth and read-only assignment retrieval. Any accessible data may be used for AI-powered features.

- AWS SES or equivalent SMTP service is configured for transactional emails (verification, password reset).
- End users have stable internet access for initial loads and synchronization; offline usage is limited to notes within the PWA.
- Browser support: Users run modern browsers (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+).
- OCR services (Google Cloud Vision, AWS Textract, or self-hosted Tesseract) are available for scanned PDF processing.

3 Functional Requirements

3.1 Feature Priority Tiers

The AI Study Vault project is structured into priority tiers:

Priority Tier	Description
Must Have (MVP)	Core features required for minimum viable product
Should Have	Important differentiators and enhancements
Nice to Have	Polish and advanced features if time permits

Table 1: Feature Priority Tiers

3.2 Functional Requirements Overview

Functional requirements are expressed as Agile user stories and formal requirement identifiers. Each requirement includes:

- **ID**: FR-#.#
- **Description**: "The system shall ..."
- **Source**: User story or stakeholder.
- **Priority**: Must Have, Should Have, or Nice to Have.

3.3 User Stories and Mapped Functional Requirements

3.3.1 User Story 1: Complete Authentication Flow

As a student, **I want to** register, verify my email, log in, and reset my password, **so that** my account is secure and I can recover access if needed.

Functional Requirements

- **FR-1.1** The system shall allow users to register with an email address and password.
- **FR-1.2** The system shall send a verification email upon registration via AWS SES or configured SMTP service.
- **FR-1.3** The system shall require email verification before full account access (optional enforcement: only if org-based sharing is enabled).
- **FR-1.4** The system shall authenticate users and issue a JWT token valid for 24 hours upon successful login.
- **FR-1.5** The system shall display a clear error message for failed login attempts.
- **FR-1.6** The system shall provide a "Forgot Password" link that sends a password reset email with a time-limited token.
- **FR-1.7** The system shall allow users to reset their password via the emailed link.
- **FR-1.8** The system shall provide a logout function that invalidates the current session.
- **FR-1.9** The system shall store passwords as bcrypt hashes with at least 10 salt rounds.

Priority: **Must Have**

3.3.2 User Story 2: User Profile and Settings

As a student, **I want to** manage my profile and settings, **so that** I can update my information and customize my experience.

Functional Requirements

- **FR-2.1** The system shall provide a settings page accessible from the main navigation.
- **FR-2.2** The system shall allow users to update their email address (with re-verification required).
- **FR-2.3** The system shall allow users to change their password (with current password confirmation).
- **FR-2.4** The system shall allow users to update their course information (university, degree course, graduation year).
- **FR-2.5** The system shall allow users to select their preferred UI language.
- **FR-2.6** The system shall allow users to delete their account with a confirmation prompt.
- **FR-2.7** The system shall implement a 7-day soft-delete period before permanent account deletion.

Priority: **Must Have**

3.3.3 User Story 3: Onboarding and Course Information

As a new student user, **I want to** provide my university and course details during signup, **so that** the platform can be personalized and enable future org-based features.

Functional Requirements

- **FR-3.1** The system shall collect university name, degree course, and graduation year during registration.
- **FR-3.2** The system shall encourage (but not require) university email addresses for potential org-based sharing features.
- **FR-3.3** The system shall store course information in the user profile for analytics and future networking features.
- **FR-3.4** The system shall display a welcome tutorial or onboarding hints after first login.

Priority: **Must Have**

3.3.4 User Story 4: Create and Edit Markdown Notes

As a student, **I want to** create and edit notes in Markdown with live preview, **so that** I can organize course materials in a structured, portable format.

Functional Requirements

- **FR-4.1** The system shall provide a split-pane editor with Markdown text on the left and live preview on the right.
- **FR-4.2** The system shall support Markdown syntax including headers, lists, code blocks, tables, and LaTeX math.
- **FR-4.3** The system shall auto-save notes to browser IndexedDB at least every 5 seconds while editing.
- **FR-4.4** The system shall support organizing notes into a folder hierarchy (e.g., `/Math/Calculus/Week-3`).
- **FR-4.5** The system shall sync notes to S3-compatible storage when online, resolving conflicts with a last-write-wins strategy.

Priority: **Must Have**

3.3.5 User Story 5: Upload and Index PDFs

As a student, **I want to** upload lecture PDFs and course materials, **so that** the system can index them for RAG chat and quiz generation.

Functional Requirements

- **FR-5.1** The system shall allow users to upload PDF files up to 50 MB (within per-user 5GB quota).
- **FR-5.2** The system shall extract text from PDFs using pdf-parse or similar library.
- **FR-5.3** The system shall support OCR for scanned PDFs via external service (Google Cloud Vision API preferred, with architecture supporting alternative backends).
- **FR-5.4** The system shall display ingestion progress as a percentage.
- **FR-5.5** The system shall split extracted text into semantic chunks (default 500 tokens with 50–100 token overlap, configurable).
- **FR-5.6** The system shall preserve metadata for each chunk, including page number and section headers.
- **FR-5.7** The system shall generate embeddings using OpenAI text-embedding-3-small (1536 dimensions) and store them in MariaDB vector columns.
- **FR-5.8** The system shall process PDF indexing as background jobs via BullMQ queues.
- **FR-5.9** The system shall display user-friendly error messages if OpenAI rate limits are hit, allowing manual retry.

Priority: **Must Have**

3.3.6 User Story 6: RAG Chat with Citations

As a student, **I want to** ask questions about my materials and receive answers with source citations, **so that** I can verify the answer and reference specific pages.

Functional Requirements

- **FR-6.1** The system shall allow users to submit free-text questions through a chat interface.
- **FR-6.2** The system shall perform hybrid search (keyword + semantic) to retrieve the top 5 relevant chunks.
- **FR-6.3** The system shall extract the text content from retrieved chunks (not vector embeddings).
- **FR-6.4** The system shall send retrieved chunk text and the question to an LLM as context to generate an answer.
- **FR-6.5** The system shall include citations in responses (page number, section name, chunk ID).
- **FR-6.6** The system shall render citations as clickable links to open the original PDF at the cited location.

- **FR-6.7** The system shall stream the LLM response token-by-token for real-time user experience.
- **FR-6.8** The system shall allow users to rate responses with thumbs up/down.
- **FR-6.9** The system shall display a user-friendly error if the LLM API is unavailable, with a manual retry option.

Priority: **Must Have**

3.3.7 User Story 7: Auto-Generate Quizzes

As a student, **I want to** generate practice quizzes from my notes or PDF sections, **so that** I can test my understanding and identify weak areas.

Functional Requirements

- **FR-7.1** The system shall allow users to select a note or document section and choose difficulty (Easy/Medium/Hard).
- **FR-7.2** The system shall generate 5–10 questions based on user configuration.
- **FR-7.3** The system shall support multiple-choice, short-answer, and true/false question types.
- **FR-7.4** The system shall provide an optional timed mode with a per-question countdown.
- **FR-7.5** The system shall display a total score after quiz completion.
- **FR-7.6** The system shall provide correct answers and explanations for each question.
- **FR-7.7** The system shall tag and track weak topics based on quiz performance.

Priority: **Must Have**

3.3.8 User Story 8: Spaced Repetition Flashcards

As a student, **I want to** use flashcard decks with SM-2 scheduling, **so that** I can review material at optimal intervals and master it efficiently.

Functional Requirements

- **FR-8.1** The system shall allow users to create flashcard decks from notes or quiz content.
- **FR-8.2** The system shall display a daily review section showing "Due Today: N cards".
- **FR-8.3** The system shall allow users to flip cards to reveal answers and rate recall on a 1–5 scale.

- **FR-8.4** The system shall update SM-2 interval and ease factor based on user ratings.
- **FR-8.5** The system shall persist flashcard state in the database and sync across devices.
- **FR-8.6** The system shall display progress metrics such as cards due, reviewed today, and mastery percentage.

Priority: **Must Have**

3.3.9 User Story 9: Offline Note Editing

As a student in a lecture without internet, **I want to** edit and create notes locally, **so that** I do not lose work and can sync automatically when online.

Functional Requirements

- **FR-9.1** The system shall be installable as a PWA.
- **FR-9.2** The system shall display an offline status indicator in the UI.
- **FR-9.3** The system shall allow full note creation and editing while offline.
- **FR-9.4** The system shall queue changes made offline for synchronization when connectivity is restored.
- **FR-9.5** The system shall automatically start synchronization when the network reconnects and show progress.
- **FR-9.6** The system shall handle write conflicts using last-write-wins or an explicit user prompt.

Priority: **Must Have**

3.3.10 User Story 10: Canvas LMS Integration

As a student, **I want to** sync my Canvas assignments and deadlines into the study calendar, **so that** I can plan my study schedule around course requirements.

Functional Requirements

- **FR-10.1** The system shall allow users to connect their Canvas account via OAuth.
- **FR-10.2** The system shall import upcoming assignments and exams with due dates.
- **FR-10.3** The system shall display imported assignments in the calendar/timetable view.
- **FR-10.4** The system shall auto-schedule study sessions before exam dates using configurable rules.
- **FR-10.5** The system shall sync LMS assignments automatically once per day via background jobs.

- **FR-10.6** The system shall provide an on-demand sync button for immediate refresh.
- **FR-10.7** The system shall display a user-friendly error if LMS sync fails, allowing manual retry.
- **FR-10.8** The system shall use any accessible Canvas API data for AI-powered planning and recommendations.

Priority: **Should Have**

3.3.11 User Story 11: Study Calendar and Timetable

As a student, **I want to** view my study schedule, flashcard reviews, and LMS deadlines in a unified calendar, **so that** I can manage my time effectively.

Functional Requirements

- **FR-11.1** The system shall provide week and month calendar views.
- **FR-11.2** The system shall display flashcard reviews due, quiz sessions, and LMS assignments on the calendar.
- **FR-11.3** The system shall auto-schedule review sessions before exams based on user-configurable rules.
- **FR-11.4** The system shall allow users to click calendar events to navigate to the associated task.
- **FR-11.5** The system shall allow users to export calendar events to iCal format.

Priority: **Should Have**

3.3.12 User Story 12: Progress Analytics Dashboard

As a student, **I want to** see my study progress and weak topics, **so that** I can identify areas needing more review.

Functional Requirements

- **FR-12.1** The system shall display study time for selected periods (weekly/monthly).
- **FR-12.2** The system shall display quiz scores over time.
- **FR-12.3** The system shall identify and rank weak topics using review counts and low scores.
- **FR-12.4** The system shall provide charts showing quiz performance and flashcard mastery progression.

Priority: **Should Have**

3.3.13 User Story 13: Full Vault Export

As a student, **I want to** export my entire vault (notes, metadata, settings), **so that** I have a portable backup and can migrate to other tools if needed.

Functional Requirements

- **FR-13.1** The system shall allow users to export their entire vault as a downloadable ZIP archive.
- **FR-13.2** The system shall include all notes in Markdown format with folder structure preserved.
- **FR-13.3** The system shall include metadata (flashcard state, quiz history, analytics data) as JSON files.
- **FR-13.4** The system shall embed export timestamp and user identifier in the archive.

Priority: **Should Have**

3.3.14 User Story 14: Quick Command Search (Cmd+K)

As a student, **I want to** quickly search and navigate using a command palette, **so that** I can access features and notes efficiently.

Functional Requirements

- **FR-14.1** The system shall support a keyboard shortcut (Cmd+K on macOS, Ctrl+K on Windows/Linux) to open a command palette.
- **FR-14.2** The system shall support fuzzy search over notes, folders, and commands (e.g., "create quiz").
- **FR-14.3** The system shall display recent items and suggestions in the command palette.
- **FR-14.4** The system shall navigate to the selected item or execute the selected command when the user presses Enter.

Priority: **Should Have**

3.3.15 User Story 15: Note Tagging and Linking

As a student, **I want to** tag notes and create bidirectional links between them, **so that** I can build a connected knowledge base and find related materials easily.

Functional Requirements

- **FR-15.1** The system shall allow users to add arbitrary tags to notes (e.g., "algorithms", "exam-prep").
- **FR-15.2** The system shall allow users to create bidirectional links between notes using wiki-style syntax (e.g., `[[Note Title]]`).
- **FR-15.3** The system shall display backlinks (notes that reference the current note) in the note view.
- **FR-15.4** The system shall provide a knowledge graph visualization showing note connections.
- **FR-15.5** The system shall support filtering notes by tags and module codes.
- **FR-15.6** The system shall suggest tags and module codes based on note content using AI.

Priority: **Should Have**

3.3.16 User Story 16: Multi-Mode Search

As a student, **I want to** search my notes using keywords or meaning-based queries, **so that** I can find information even when I don't remember exact terms.

Functional Requirements

- **FR-16.1** The system shall support keyword search using FULLTEXT indexes for fast exact matching.
- **FR-16.2** The system shall support semantic search using vector embeddings for meaning-based retrieval.
- **FR-16.3** The system shall support hybrid search combining keyword and semantic results.
- **FR-16.4** The system shall allow filtering search results by module, tags, document type, semester, and academic year.
- **FR-16.5** The system shall display search results with relevance scores and preview snippets.

Priority: **Must Have**

3.3.17 User Story 17: Share Materials with Study Groups

As a student, **I want to** share my note collections with classmates, **so that** study groups can collaborate (read-only, with attribution).

Functional Requirements

- **FR-17.1** The system shall allow users to generate shareable read-only links for note folders.
- **FR-17.2** The system shall include original author attribution on shared materials.
- **FR-17.3** The system shall allow recipients to copy shared materials into their own vault.
- **FR-17.4** The system shall allow authors to revoke share links, removing access.
- **FR-17.5** The system shall track metadata for which materials are shared with which tokens.
- **FR-17.6** (Optional) If org-based sharing is implemented, the system shall require email verification and enable sharing within university organizations.

Priority: **Nice to Have**

3.3.18 User Story 18: Internationalization (i18n) Support

As a non-English-speaking student, **I want to** use the platform in my native language, **so that** I can navigate and understand the interface comfortably.

Functional Requirements

- **FR-18.1** The system shall be structured to support multiple UI languages using i18next or similar framework.
- **FR-18.2** The system shall provide a language selector in user settings.
- **FR-18.3** The system shall externalize all UI strings (navigation, buttons, form labels, error messages) for translation.
- **FR-18.4** The system shall not modify user-generated content (notes, PDFs); they remain in the original language.
- **FR-18.5** The system shall default LLM responses to the language of the user's query where possible.

Priority: **Nice to Have**

3.3.19 User Story 19: Export Individual Notes/Data

As a student, **I want to** export individual notes to PDF and analytics data to CSV, **so that** I can share or archive specific materials.

Functional Requirements

- **FR-19.1** The system shall allow exporting a single note or folder to PDF.
- **FR-19.2** The system shall preserve Markdown formatting (headers, lists, code blocks, LaTeX math) in the exported PDF.
- **FR-19.3** The system shall include a table of contents for multi-note exports.
- **FR-19.4** The system shall allow export of analytics data (study time, quiz scores, weak topics) as CSV.

Priority: **Nice to Have**

3.4 RAG Pipeline Architecture

The system implements a standard industry RAG pipeline:

1. **Preprocessing**: PDFs are uploaded to S3, text extracted via pdf-parse, chunked into 500-token segments with 50–100 token overlap, and embedded using OpenAI text-embedding-3-small (1536 dimensions).
2. **Storage**: Binary PDFs remain in S3; extracted text and vector embeddings are stored in MariaDB for search.
3. **Retrieval**: User queries trigger both keyword (FULLTEXT, 5ms) and semantic (vector similarity, 50ms) searches. Combined results return in <100ms.
4. **Augmentation**: Retrieved chunks' *text content* (not vector embeddings) is injected into the LLM prompt as context.
5. **Generation**: The LLM generates answers grounded in the provided text chunks, with citations to source pages.

Key Insight: Vector embeddings are used only for similarity-based retrieval. The LLM receives and processes actual text content, ensuring answers are grounded in source material and citations are accurate.

This architecture follows proven patterns used by Notion AI, ChatGPT document retrieval, and Google Drive search, with scalability via MariaDB HNSW indexes handling millions of vectors efficiently.

3.5 Functional Summary

The system supports three primary workflows:

1. **Note + PDF Ingestion**: Upload materials, generate embeddings with 500-token chunks and 50–100 token overlap, store in MariaDB with native vector support.
2. **RAG Learning**: Ask questions, retrieve context via hybrid keyword+semantic search, send retrieved text chunks (not vectors) to LLM as context, generate cited answers.
3. **Adaptive Review**: Auto-generate quizzes, track weak areas, schedule flashcards via SM-2.

4 Non-Functional Requirements

Non-functional requirements are organized by quality attribute categories and use identifiers of the form NFR-*Category*-#.

4.1 Performance

- **NFR-P-1** RAG query end-to-end response time shall be ≤ 3 seconds under nominal load.
- **NFR-P-2** PDF indexing time shall be ≤ 2 minutes for a 50 MB file.
- **NFR-P-3** Keyword search shall complete in ≤ 5 ms; semantic vector search shall complete in ≤ 50 ms; hybrid search shall complete in ≤ 100 ms.
- **NFR-P-4** Quiz generation shall complete in ≤ 5 seconds.
- **NFR-P-5** Note synchronization of a 500 KB note shall complete in ≤ 2 seconds under normal network conditions.
- **NFR-P-6** Frontend cold load shall be ≤ 4 seconds; cached load shall be ≤ 2 seconds on a typical student laptop and connection.
- **NFR-P-7** Each backend service shall support at least 10 concurrent requests without significant degradation.

4.2 Security

- **NFR-S-1** All APIs, except `/login`, `/register`, and `/verify-email`, shall require a valid JWT for access.
- **NFR-S-2** Passwords shall be hashed using bcrypt with a salt factor ≥ 10.
- **NFR-S-3** All external communications shall use TLS 1.2 or higher (HTTPS only).
- **NFR-S-4** CORS policy shall restrict access to approved frontend origins.
- **NFR-S-5** The system shall implement rate limiting on public endpoints with clear error messages when limits are exceeded.
- **NFR-S-6** Database credentials and API keys shall be stored only in environment variables or secret stores and shall be encrypted at rest where supported.
- **NFR-S-7** Pre-signed S3 URLs shall have a maximum validity of 1 hour.
- **NFR-S-8** Vector search queries shall always be filtered by user identifier to prevent cross-user data leakage.
- **NFR-S-9** Password reset tokens shall expire after 1 hour and be single-use only.
- **NFR-S-10** Email verification tokens shall expire after 24 hours.

4.3 Reliability & Availability

- **NFR-R-1** System availability shall be at least 99% during the grading window.
- **NFR-R-2** Database backups shall run automatically (AWS RDS automated backups for production, manual dumps for development).
- **NFR-R-3** Asynchronous jobs (PDF indexing, embedding, LMS sync) shall be retried up to 3 times with exponential backoff on failure.
- **NFR-R-4** In case of OpenAI API unavailability or rate limiting, the system shall display a user-friendly error with manual retry option (no complex automatic retry logic for MVP).
- **NFR-R-5** User sessions shall survive restarts of application services by persisting necessary state in the database.
- **NFR-R-6** The PWA shall attempt to reconnect within 30 seconds of a network loss and resume synchronization.

4.4 Usability

- **NFR-U-1** The UI shall follow a modern, clean design consistent with Material Design 3 or similar design system.
- **NFR-U-2** A new user shall be able to complete registration, upload a PDF, and ask at least one question in ≤ 5 minutes without external help.
- **NFR-U-3** Forms shall provide real-time validation feedback, including password strength indicators.
- **NFR-U-4** The UI shall support keyboard navigation using Tab, Enter, and Escape keys for primary flows.
- **NFR-U-5** The application shall provide a responsive layout optimized for desktops and tablets (mobile phones are not optimized in MVP).
- **NFR-U-6** Error messages shall be actionable and non-technical, providing guidance on how to resolve the issue.
- **NFR-U-7** Dark mode shall be the default and only theme for MVP.

4.5 Maintainability

- **NFR-M-1** All JavaScript/TypeScript code shall comply with ESLint and Prettier rules enforced in CI.
- **NFR-M-2** All public APIs shall be documented using OpenAPI 3.1 specification, either auto-generated from code annotations or manually maintained.
- **NFR-M-3** Database schema changes shall be managed using version-controlled migrations (e.g., Prisma Migrate).

- **NFR-M-4** The system shall emit structured JSON logs including request IDs and timestamps.
- **NFR-M-5** The root README shall describe setup, development workflow, and deployment steps required to run the system.
- **NFR-M-6** Automated dependency scanning (e.g., Dependabot or Snyk) shall be enabled to detect critical vulnerabilities.

5 External Interface Requirements

5.1 User Interfaces

5.1.1 Authentication Pages

- Registration form: Email, password (with strength indicator), university, degree course, graduation year
- Login form: Email, password, "Forgot password?" link
- Email verification page: Success/failure messaging
- Password reset form: New password entry with confirmation

5.1.2 Settings Page

- Profile section: Update email (with re-verification), change password, edit course information
- Preferences: Language selector, theme (dark mode only for MVP)
- Account management: Storage usage display, delete account button with confirmation

5.1.3 Editor Interface

- Markdown editor with live split-pane preview.
- Left sidebar: Folder/note tree view.
- Top navigation: Upload PDF, RAG Chat, Create Quiz, Flashcards, Analytics, Settings.
- Status bar: Offline indicator, sync progress, word count, storage usage.
- Command palette: Cmd+K/Ctrl+K for quick navigation.

5.1.4 RAG Chat Panel

- Scrollable message history with timestamps.
- Input field: "Ask a question about your materials..."
- Response: Formatted text with inline citations (clickable page references).
- Feedback buttons: Thumbs up/down for quality rating.
- Error handling: Clear message if LLM unavailable with retry button.

5.1.5 Quiz Interface

- One question per screen.
- Progress bar: "Question X of Y".
- Multiple-choice: Radio buttons; short-answer: text input field.
- Timed mode: Optional countdown timer.
- Results: Score, question-by-question breakdown, weak topics highlighted.

5.1.6 Flashcard Interface

- Card display: Front (prompt) and back (answer) with flip animation.
- Response buttons: 1–5 (Forget / Poor / Good / Very Good / Perfect).
- Progress indicators: "Due today: N. Reviewed: M."

5.1.7 Calendar/Timetable View

- Week/month calendar grid.
- Color-coded events: Flashcard reviews (blue), LMS assignments (red), study sessions (green).
- Click event to open details or start associated task.
- Manual sync button for LMS updates.

5.2 API Interface

Core REST API endpoints:

- **POST /auth/register**: Create user account with email, password, and course info; sends verification email; returns JWT token.
- **GET /auth/verify-email**: Verify email with token from link.
- **POST /auth/login**: Authenticate credentials, returns JWT token and user metadata.

- **POST /auth/forgot-password**: Send password reset email.
- **POST /auth/reset-password**: Reset password using token from email.
- **POST /auth/logout**: Invalidate current JWT session.
- **GET /user/profile**: Retrieve user profile and settings.
- **PUT /user/profile**: Update email, password, or course information.
- **DELETE /user/account**: Soft-delete user account (7-day grace period).
- **POST /chat**: Accepts query and document identifiers, returns answer, citations, and sources.
- **GET /documents**: Returns list of user documents with id, title, and page count.
- **POST /documents**: Upload and register a new PDF document.
- **POST /quizzes**: Creates a quiz from a document, number of questions, and difficulty.
- **POST /quizzes/{id}/submit**: Submits user answers and returns score and feedback.
- **GET /flashcards/due**: Returns cards due for review (id, front, back).
- **POST /flashcards/{id}/review**: Accepts rating and returns updated interval and next due date.
- **GET /calendar/events**: Returns flashcard reviews, LMS deadlines, and study sessions.
- **POST /calendar/sync**: Trigger on-demand LMS sync.
- **GET /analytics/dashboard**: Returns study time, quiz scores, and weak-topic data.
- **POST /vault/export**: Exports entire vault (notes, metadata, settings) as ZIP archive.
- **POST /notes/share**: Generates a shareable link for a note or folder.
- **GET /notes/search**: Performs keyword, semantic, or hybrid search with filters.
- **POST /notes/{id}/tags**: Adds or removes tags from a note.
- **POST /notes/{id}/links**: Creates bidirectional links between notes.

5.3 Database Interface

5.3.1 Storage Architecture

The system employs a hybrid storage model:

- **MariaDB**: Stores metadata, searchable content, embeddings, and relational data.
- **S3-Compatible Storage**: Stores full Markdown files and uploaded PDFs.

This architecture minimizes database size, maintains query performance, and reduces storage costs.

5.3.2 Data Entities and Relationships

The system shall store the following data entities:

- **User Accounts**: Authentication credentials, profile information, course details (university, degree, graduation year), storage quotas, and preferences.
- **Notes**: Markdown content, folder organization, academic metadata (module codes, document types, semester), and searchable text extracts.
- **Vector Embeddings**: Semantic embeddings for notes and PDF chunks to enable similarity-based retrieval, with associated metadata (page numbers, section headers).
- **Tags and Links**: User-defined tags for categorization and bidirectional wiki-style links between notes for knowledge graph construction.
- **Documents (PDFs)**: Uploaded file metadata, S3 storage paths, page counts, and indexing status.
- **Quizzes and Flashcards**: Quiz questions with answers and explanations, flashcard front/back content, SM-2 scheduling parameters (interval, ease factor, next due date).
- **Calendar Events**: LMS assignments, flashcard review sessions, and user-scheduled study blocks with due dates and sources.
- **Sharing Permissions**: Access control for shared notes and folders, including permission levels, expiration dates, and revocation status.
- **Authentication Tokens**: Password reset tokens and email verification tokens with expiration and usage tracking.
- **Analytics Data**: Study time, quiz performance, weak topics, and progress metrics.

5.3.3 Database Capabilities

The database shall support the following capabilities:

- **FULLTEXT Search**: Fast keyword search on note content with natural language matching (5ms query time).
- **Vector Similarity Search**: Approximate nearest neighbor search on embeddings using MHNSW or similar indexing for semantic retrieval (50ms query time).
- **Hybrid Search**: Combined keyword and semantic search with configurable weighting (<100ms query time).
- **User-Scoped Queries**: All queries shall be filtered by user identifier to enforce row-level security and prevent cross-user data leakage.
- **Filtering and Faceting**: Support for filtering search results and content by tags, module codes, document types, academic year, and semester.
- **Transactional Integrity**: ACID-compliant operations for critical workflows (user registration, payment processing, account deletion).
- **Schema Migrations**: Version-controlled database schema changes to support iterative development.

5.4 External Services

- **OpenAI API**: Embeddings (text-embedding-3-small, 1536 dims) and chat completion (GPT-3.5/GPT-4).
- **S3-Compatible Object Storage**: For notes, PDFs, and exported documents (e.g., AWS S3, Backblaze B2, Wasabi).
- **Redis**: Caching and BullMQ job queue backend.
- **MariaDB**: Unified database (relational + native vector search).
- **Canvas LMS API**: OAuth and assignment sync (read-only); any accessible data may be used for AI features.
- **Google Cloud Vision API**: OCR for scanned PDFs (preferred); architecture supports alternative backends.
- **AWS SES**: Transactional emails (verification, password reset) via configured SMTP service.
- **Authentication**: JWT-based; no external SSO required for MVP.

6 Data & Legal Requirements

6.1 Data Handling

- Data at rest in S3 shall be encrypted using server-side encryption or KMS keys where available.
- AWS RDS MariaDB instances shall have storage encryption enabled.
- All data in transit shall be encrypted using TLS 1.2+.
- User data shall be retained until account deletion; soft-delete shall keep data for 7 days before hard deletion.
- Users shall be able to export all notes and metadata as a downloadable vault archive.
- Account deletion shall remove notes, embeddings, and quiz history within 7 days.
- Password reset and email verification tokens shall be hashed before storage and expired appropriately.

6.2 Compliance

- **GDPR**: Provide data subject rights for EU users (access, deletion, portability) within project constraints.
- **FERPA**: If integrated with institutional LMS data in future, student records shall be treated as sensitive educational records.
- **MIT License Compliance**: Notea attribution is included prominently in the project documentation and UI credits.
- **Copyright**: Users are responsible for uploaded content; the system provides attribution tracking for shared materials.

6.3 Audit & Logging

- API access logs shall include timestamp, user ID (if available), endpoint, and status code.
- Document upload logs shall include file size, hash, user, and timestamp.
- Authentication events (login, logout, password reset, email verification) shall be logged.
- Deletion operations shall be logged with timestamp, user, and content hash for recovery within the soft-delete window.
- Logs shall be retained for at least 90 days for security review.

7 Verification and Acceptance

7.1 Testing Strategy

- **Unit Tests**: Cover core business logic (SM-2 algorithm, semantic chunking, embedding generation) with 70%+ code coverage.
- **Integration Tests**: Test API endpoints with mock or test database instances.
- **End-to-End Tests**: Critical user flows (register → verify email → upload PDF → ask question → receive cited answer).
- **Manual QA**: Create 3–5 test user accounts and thoroughly test all features with realistic usage patterns.
- **Performance Testing**: Lighthouse audits for frontend performance, used to justify improvements and optimizations.
- **Test Data**: Prepare sample PDFs, notes, and seed data for consistent development and demo environments.
- **Optional Staging Environment**: AWS staging environment for pre-production validation before deploying to production.

7.2 Acceptance Criteria

The system is considered acceptable when:

1. All "Must Have" functional requirements are implemented and verified.
2. Non-functional requirements (performance, security, reliability) are met for representative workloads.
3. `docker compose up` works on a fresh environment without manual intervention.
4. End-to-end demo workflow completes successfully: register, verify email, upload PDF, index, ask RAG question, generate quiz, review flashcard.
5. Production deployment via AWS Amplify auto-deploy from `prod` branch is operational.
6. Code coverage for core logic is $\geq 70\%$.
7. Lighthouse performance scores meet acceptable thresholds (informative, not blocking).
8. Academic stakeholder approves the deliverable.

7.3 Deployment and CI/CD

- **Local Development**: `docker compose up` starts all services (MariaDB, Redis, backend API, frontend, workers).
- **Continuous Integration**: Automated tests, linting (ESLint/Prettier), and OpenAPI spec generation/validation run on push to feature or `dev` branches.
- **Production Deployment**: AWS Amplify auto-deploys from `prod` branch after successful CI checks.
- **Branch Strategy**: Feature branches → `dev` (manual audit/promotion) → `prod` (auto-deploy).
- **OpenAPI Documentation**: Either auto-generated from code annotations using available tooling, or manually maintained as OpenAPI 3.1 specification files.
- **Optional Staging**: AWS staging environment for pre-production testing before merging to `prod`.

Appendix A: Glossary

- **RAG (Retrieval-Augmented Generation)**: A technique combining vector search with large language models. Vector embeddings are used to retrieve relevant text chunks via similarity search; these chunks are then provided as context in the LLM prompt to generate grounded, cited answers.
- **PWA (Progressive Web App)**: A web application that can be installed and used offline.
- **SM-2 (SuperMemo 2)**: A spaced repetition algorithm for optimizing flashcard review intervals.
- **LMS (Learning Management System)**: Platforms like Canvas, Moodle, or Blackboard used by universities.
- **JWT (JSON Web Token)**: A standard for secure authentication tokens.
- **Vector Embedding**: A numerical representation of text (e.g., 1536-dimensional) used for semantic similarity search via cosine distance.
- **Chunking**: The process of splitting documents into smaller segments (typically 500 tokens with 50–100 token overlap) for embedding and retrieval.
- **MHNSW (Hierarchical Navigable Small World)**: A graph-based approximate nearest neighbor search algorithm for vector similarity.
- **OCR (Optical Character Recognition)**: Technology for extracting text from scanned images or PDFs.
- **MariaDB**: An open-source relational database with native vector search support.

- **Semantic Search**: Search based on meaning rather than exact keyword matching, using vector embeddings (50ms).
- **Keyword Search**: Traditional FULLTEXT search for exact term matching (5ms).
- **Hybrid Search**: Combines keyword (FULLTEXT) and semantic (vector) search results with weighted scoring (<100ms).
- **Bidirectional Links**: Wiki-style connections between notes where both source and target notes display the relationship.
- **AWS SES (Simple Email Service)**: Amazon's email sending service for transactional emails like verification and password resets.

Appendix B: Use Cases

Use Case 1: First-Time User Onboarding

Actor: New student user
Preconditions: None
Flow:

1. User navigates to the application URL.
2. User clicks "Sign Up" and enters email, password, university, degree course, and graduation year.
3. System validates input, creates account, and sends verification email via AWS SES.
4. User clicks verification link in email; system confirms and enables full access.
5. User is logged in and sees a welcome screen with tutorial hints.
6. User clicks "Upload PDF", selects a lecture PDF, and uploads.
7. System processes PDF in background (extracts text with pdf-parse, creates 500-token chunks with 50-token overlap, generates embeddings via OpenAI, stores in MariaDB) and displays progress.
8. Once indexed, user opens RAG chat and asks: "What are the key topics in this lecture?"
9. System performs hybrid keyword+semantic search, retrieves top 5 chunk texts, and sends them to GPT-4 as context.
10. System streams response with citations to page numbers.

Postconditions: User has verified account, uploaded document, and asked their first question.

Use Case 2: Password Reset

Actor: Returning user who forgot password
Preconditions: User has an existing account
Flow:

1. User navigates to login page and clicks "Forgot Password?"
2. User enters their email address.
3. System sends password reset email via AWS SES with time-limited token (expires in 1 hour).
4. User clicks reset link in email.
5. System displays password reset form.
6. User enters and confirms new password.
7. System validates password strength, updates password hash, and invalidates the token.
8. User is redirected to login page with success message.

Postconditions: User password is reset and user can log in with new credentials.

Use Case 3: Daily Review Routine

Actor: Returning student user
Preconditions: User has created flashcards and configured study schedule.
Flow:

1. User opens the app.
2. Dashboard shows: "Due Today: 12 flashcards, 1 quiz scheduled."
3. User clicks "Start Review".
4. System shows flashcard prompt; user flips and rates (1–5).
5. System updates SM-2 interval and proceeds to next card.
6. After completing flashcards, user clicks "Take Quiz".
7. System generates quiz from a selected document section.
8. User completes quiz and views score and weak topics.
9. System updates analytics dashboard with today's progress.

Postconditions: Flashcards reviewed, quiz completed, progress tracked.

Use Case 4: LMS Integration and Manual Sync

Actor: Student user with Canvas account
Preconditions: User has authorized Canvas OAuth
Flow:

1. System automatically syncs Canvas assignments once per day via background job.
2. User opens calendar view and sees upcoming assignments imported from Canvas.
3. User notices a new assignment was just posted but not yet visible.
4. User clicks "Sync Now" button.
5. System triggers on-demand LMS sync via `POST /calendar/sync`.
6. If sync succeeds, new assignment appears in calendar.
7. If sync fails (e.g., Canvas API unavailable), system displays user-friendly error: "Unable to sync with Canvas. Please try again later."
8. User can click "Retry" to attempt sync again.

Postconditions: Calendar is updated with latest LMS data or user is informed of error.

Use Case 5: Building a Knowledge Graph

Actor: Student organizing course materials
Preconditions: User has multiple notes on related topics.
Flow:

1. User creates a note on "Binary Search Trees" and adds tags: "algorithms", "data-structures", "exam-prep".
2. User references another note using wiki-style syntax: `[[Sorting Algorithms]]`.
3. System creates a bidirectional link between "Binary Search Trees" and "Sorting Algorithms".
4. User clicks "View Graph" to see knowledge graph visualization.
5. System displays nodes (notes) and edges (links) showing how topics connect.
6. User clicks a node in the graph to navigate directly to that note.
7. System suggests the tag "algorithms" for a new untitled note based on content analysis.

Postconditions: Notes are linked, tagged, and visualized as a connected knowledge base.